



ABSTRACCIÓN Y USO DE DATOS
3CM30

Reporte de prácticas

Murata Abreu Rodrigo Yael
Van Scoit Urtiz Ian Aaron

INDICE

1. HOLA MUNDO	3
2. SUMA DE DOS VALORES COMO PARÁMETROS.....	3
3. CALCULADORA BÁSICA (SUMA, RESTA, DIVISIÓN, MULTIPLICACIÓN)	3
4. CALCULADORA CON 2, 0 Y 3 PARÁMETROS.....	4
5. CALCULADORA NUEVA CON HERENCIA (SUMA, RESTA, DIV, MULT, POT, MOD).....	4
6. CALCULADORA CON HERENCIA Y SOBRESERITURA DE FUNCIONES	4
7. CREACIÓN DE TIPO DE DATO EN PE Y POO	5
8. IDENTIFICAR EL TAMAÑO EN PE Y POO.....	5
9. PROMEDIO	5
10. PROMEDIO CON ARREGLO	6
11. PROMEDIO CON ARREGLO Y PUNTERO	6
12. MULTIPLICACIÓN DE MATRICES	6
13. LISTA DE PERSONAS Y COCHES	6
14. LISTA DE PERSONAS CON PUNTERO AL ARREGLO	7
15. FACTORIAL Y SERIE DE FIBONACCI CON FUNCIONES FIBO() Y FIBO2()	7
16. FRACTALES: TRIÁNGULO DE SIERPINSKI Y POLVO DE CANTOR	7
17. PILA CON ARREGLOS (DATO BASE)	8
18. PILA CON ARREGLOS (NUEVO TIPO DE DATO).....	8
19. PILA CON PUNTEROS (DATO BASE).....	8
20. PILA CON PUNTEROS (NUEVO TIPO DE DATO)	9
21. PILA CON LIBRERÍAS (DATO BASE).....	9
22. PILA CON LIBRERÍAS (NUEVO TIPO DE DATO)	9
23. COLA CON ARREGLOS (DATO BASE).....	9
24. COLA CON ARREGLOS (NUEVO TIPO DE DATO).....	10
25. COLA CON PUNTEROS (DATO BASE).....	10
26. COLA CON PUNTEROS (NUEVO TIPO DE DATO)	10
27. COLA CON LIBRERÍAS (DATO BASE).....	10
28. COLA CON LIBRERÍAS (NUEVO TIPO DE DATO)	10
29. FILA CON ARREGLOS (DATO BASE)	11
30. FILA CON ARREGLOS (NUEVO TIPO DE DATO).....	11
31. FILA CON PUNTEROS (DATO BASE).....	11
32. FILA CON PUNTEROS (NUEVO TIPO DE DATO)	11
33. FILA CON LIBRERÍAS (DATO BASE).....	11
34. FILA CON LIBRERÍAS (NUEVO TIPO DE DATO)	12
35. ORDENAMIENTO BUBBLE SORT.....	12
36. ORDENAMIENTO MERGE SORT	12
37. ORDENAMIENTO QUICK SORT.....	12
38. IMPLEMENTACIÓN DE GRAFO (NO DIRIGIDO).....	13
39. IMPLEMENTACIÓN DE DÍGRAFO (DIRIGIDO)	15
40. IMPLEMENTACIÓN DE ÁRBOL BST.....	18

PRIMER PARCIAL

1. HOLA MUNDO

El siguiente código se realizó siguiendo la estructura de archivos solicitada por el docente, compuesta por tres componentes: Programa Principal (main.cpp), Programa de Funciones (archivo.cpp) y Librería (archivo.h).

Para esta práctica se definió una clase llamada Saludo y dentro de ella una función denominada Hola. Se empleó una variable de tipo string con el fin de almacenar el nombre del usuario y posteriormente mostrarlo con un mensaje de bienvenida.

Durante el desarrollo se presentó un error al crear los archivos de forma independiente en lugar de integrarlos como parte de un mismo proyecto. Una vez identificado el problema, se realizó la corrección correspondiente y la compilación y ejecución del programa se llevaron a cabo exitosamente.

La salida del programa se exportó en los cuatro formatos requeridos: salida.txt, salida.csv, salida.json y salida.xml.

2. SUMA DE DOS VALORES COMO PARÁMETROS

En esta práctica se implementó una función que recibe dos valores numéricos como parámetros y retorna su suma. La estructura del proyecto mantiene la separación entre la definición de la interfaz en el archivo de cabecera (.h), la implementación de la lógica en el archivo de funciones (.cpp) y el punto de entrada en main.cpp.

Se trabajó con tipos de datos enteros y de punto flotante para validar el comportamiento de la función ante distintas entradas. Los parámetros son pasados directamente desde la función principal al momento de llamar a la función de suma.

El programa fue compilado y ejecutado sin errores. Los resultados obtenidos fueron exportados en los cuatro formatos de salida: salida.txt, salida.csv, salida.json y salida.xml.

3. CALCULADORA BÁSICA (SUMA, RESTA, DIVISIÓN, MULTIPLICACIÓN)

Este proyecto consiste en el desarrollo de una calculadora básica capaz de realizar las cuatro operaciones aritméticas fundamentales: suma, resta, división y multiplicación. Cada operación fue implementada como una función independiente dentro del archivo de funciones.

Se consideraron casos especiales como la división entre cero, para los cuales se implementó una validación que impide la ejecución de la operación y notifica al usuario del error.

La interacción con el programa se realiza ingresando dos operandos y seleccionando la operación deseada. El resultado es mostrado en pantalla y exportado en los formatos salida.txt, salida.csv, salida.json y salida.xml.

4. CALCULADORA CON 2, 0 Y 3 PARÁMETROS

En esta práctica se amplió el concepto de calculadora para manejar distintos números de parámetros: funciones con dos operandos, funciones sin operandos (que emplean valores predefinidos) y funciones con tres operandos.

Se aplicó el concepto de sobrecarga de funciones, permitiendo que una misma función pueda comportarse de manera diferente dependiendo de la cantidad y tipo de argumentos recibidos. Esto proporciona mayor flexibilidad al programa sin sacrificar legibilidad.

El proyecto fue validado con múltiples pruebas de entrada para verificar el correcto funcionamiento de cada variante. Los resultados se exportaron en los cuatro formatos de salida establecidos.

5. CALCULADORA NUEVA CON HERENCIA (SUMA, RESTA, DIV, MULT, POT, MOD)

Esta práctica introduce el concepto de herencia dentro de la Programación Orientada a Objetos. Se creó una clase base con las operaciones aritméticas básicas (suma, resta, multiplicación y división) y a partir de ella se derivó una clase hija que extiende las funcionalidades incorporando las operaciones de potencia y módulo.

La herencia permite reutilizar el código existente sin necesidad de reescribirlo, reduciendo la redundancia y facilitando el mantenimiento del sistema. La clase derivada hereda todos los métodos de la clase base y añade los propios.

Se realizaron pruebas con distintos valores para verificar el correcto funcionamiento de las seis operaciones. Los resultados fueron exportados en los formatos: salida.txt, salida.csv, salida.json y salida.xml.

6. CALCULADORA CON HERENCIA Y SOBRESCRITURA DE FUNCIONES

Continuando con el paradigma de herencia, en esta práctica se implementó la sobrescritura de métodos (override). La clase derivada redefine las funciones heredadas de la clase base para modificar su comportamiento, conservando la misma firma de método.

La sobrescritura permite que objetos de la clase derivada ejecuten su propia versión de un método cuando éste es invocado, incluso a través de un puntero a la clase base, gracias al polimorfismo.

Se verificó que las funciones sobrescritas efectivamente reemplazaran el comportamiento original. Las salidas del programa fueron exportadas en los cuatro formatos requeridos.

7. CREACIÓN DE TIPO DE DATO EN PE Y POO

En esta práctica se abordó la definición de tipos de dato personalizados tanto bajo el enfoque de Programación Estructurada (PE) como de Programación Orientada a Objetos (POO). En el enfoque estructurado se utilizaron estructuras (struct) para agrupar datos relacionados, mientras que en POO se emplearon clases con sus respectivos atributos y métodos.

Se compararon ambos enfoques para analizar las diferencias en la forma de definir, instanciar y manipular los tipos de dato creados. La POO ofrece encapsulamiento, lo cual permite controlar el acceso a los datos a través de métodos públicos y privados.

Los resultados de ambas implementaciones fueron exportados en los formatos: salida.txt, salida.csv, salida.json y salida.xml.

8. IDENTIFICAR EL TAMAÑO EN PE Y POO

Esta práctica consistió en identificar y comparar el tamaño en memoria de distintos tipos de datos, tanto en la Programación Estructurada como en la Programación Orientada a Objetos. Se utilizó el operador sizeof() para determinar el espacio que ocupa cada tipo en bytes.

Se analizaron tipos de dato primitivos (int, float, double, char) y también estructuras y clases definidas por el usuario. Este análisis es fundamental para entender el uso eficiente de memoria en aplicaciones de bajo nivel.

Los datos obtenidos fueron presentados de forma comparativa y exportados en los cuatro formatos de salida requeridos.

9. PROMEDIO

En esta práctica se implementó un programa para calcular el promedio de un conjunto de valores numéricos ingresados por el usuario. La lógica fue encapsulada en una función que recibe los datos, realiza la suma acumulativa y divide entre la cantidad de elementos.

Se validó que el número de elementos sea mayor a cero para evitar una división entre cero. El programa permite al usuario ingresar tantos valores como desee antes de calcular el resultado final.

La salida con el promedio calculado fue exportada correctamente en los formatos: salida.txt, salida.csv, salida.json y salida.xml.

10. PROMEDIO CON ARREGLO

Esta práctica amplía la práctica anterior al utilizar un arreglo estático para almacenar los valores antes de calcular el promedio. El uso de arreglos permite conservar los datos ingresados para su posterior procesamiento o consulta.

Se definió un tamaño fijo para el arreglo y se llenó con los valores proporcionados por el usuario. Posteriormente se realizó el cálculo del promedio iterando sobre todos los elementos almacenados.

Los resultados fueron exportados en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.

11. PROMEDIO CON ARREGLO Y PUNTERO

En esta versión del cálculo de promedio se incorporó el uso de punteros para acceder y recorrer los elementos del arreglo. El puntero apunta a la dirección del primer elemento y mediante aritmética de punteros se recorre la estructura.

El uso de punteros permite una mayor eficiencia en el acceso a memoria y prepara al programador para el manejo de estructuras de datos dinámicas. Se validó que los resultados fueran idénticos a los obtenidos con la versión sin punteros.

La salida fue exportada en los cuatro formatos requeridos: salida.txt, salida.csv, salida.json y salida.xml.

12. MULTIPLICACIÓN DE MATRICES

Esta práctica implementa la multiplicación de dos matrices de dimensiones compatibles. Se utilizaron arreglos bidimensionales para representar las matrices y se implementó el algoritmo estándar de multiplicación mediante tres ciclos anidados.

El programa solicita al usuario las dimensiones y los valores de ambas matrices, valida que las dimensiones sean compatibles para la multiplicación y presenta el resultado en forma de matriz.

El resultado de la operación fue exportado en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.

13. LISTA DE PERSONAS Y COCHES

En esta práctica se definieron tipos de dato personalizados para representar personas y coches, cada uno con sus propios atributos. Se crearon listas de ambos tipos y se implementaron funciones para agregar, mostrar y eliminar elementos de dichas listas.

Esta práctica introduce el concepto de manejo de colecciones de objetos complejos, lo cual sienta las bases para el estudio de estructuras de datos más avanzadas como pilas, colas y listas enlazadas.

Los datos de las listas fueron exportados en los cuatro formatos de salida: salida.txt, salida.csv, salida.json y salida.xml.

14. LISTA DE PERSONAS CON PUNTERO AL ARREGLO

Esta práctica extiende la anterior al acceder a la lista de personas mediante punteros. En lugar de trabajar directamente con el arreglo, se utiliza un puntero que apunta al inicio del arreglo y se recorre mediante aritmética de punteros.

El uso de punteros permite comprender la relación entre arreglos y memoria dinámica, preparando el camino hacia estructuras como listas enlazadas. Se verificó que el acceso mediante puntero produjera los mismos resultados que el acceso directo.

Los resultados fueron exportados en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.

15. FACTORIAL Y SERIE DE FIBONACCI CON FUNCIONES FIBO() Y FIBO2()

Esta práctica implementa dos algoritmos clásicos de programación: el factorial de un número y la serie de Fibonacci. Para el cálculo del factorial se empleó tanto la versión iterativa como la recursiva, permitiendo comparar el comportamiento de ambos enfoques.

Para la serie de Fibonacci se implementaron dos versiones: fibo(), que utiliza recursión simple, y fibo2(), que emplea programación dinámica o memoización para optimizar el cálculo evitando recalcular valores ya obtenidos.

Los resultados de ambas funciones fueron exportados en los cuatro formatos de salida: salida.txt, salida.csv, salida.json y salida.xml.

16. FRACTALES: TRIÁNGULO DE SIERPINSKI Y POLVO DE CANTOR

En esta práctica se implementaron dos fractales matemáticos clásicos mediante recursión. El Triángulo de Sierpinski se construye dividiendo un triángulo equilátero en cuatro triángulos más pequeños y eliminando el central de forma recursiva.

El Polvo de Cantor se genera dividiendo un segmento de línea en tres partes iguales y eliminando la parte central, repitiendo el proceso en los segmentos restantes. Ambas construcciones ilustran la naturaleza recursiva de los fractales.

Las representaciones generadas fueron exportadas en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.

SEGUNDO PARCIAL

17. PILA CON ARREGLOS (DATO BASE)

En esta práctica se implementó la estructura de datos Pila (Stack) utilizando un arreglo estático y tipos de dato primitivos como dato base. La pila sigue el principio LIFO (Last In, First Out), donde el último elemento en ingresar es el primero en ser extraído.

Se implementaron las operaciones fundamentales: `push()` para insertar un elemento en la cima, `pop()` para extraer el elemento de la cima, `peek()` para consultar el elemento superior sin extraerlo, y funciones auxiliares para verificar si la pila está llena o vacía.

Se validaron los casos límite de desbordamiento (overflow) y vaciado (underflow). Los resultados fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

18. PILA CON ARREGLOS (NUEVO TIPO DE DATO)

Esta práctica replica la implementación de la Pila con arreglos, pero en lugar de utilizar un dato primitivo como tipo base, se emplea un tipo de dato personalizado (struct o clase). Esto permite almacenar objetos más complejos con múltiples atributos dentro de la pila.

Se adaptaron todas las operaciones de la pila para trabajar con el nuevo tipo de dato, asegurando que la inserción y extracción de elementos complejos funcionen correctamente.

Los resultados fueron exportados en los cuatro formatos de salida: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

19. PILA CON PUNTEROS (DATO BASE)

En esta práctica se implementó la Pila utilizando memoria dinámica y punteros, en lugar de un arreglo estático. Cada nodo de la pila es creado dinámicamente con `new` y contiene un dato y un puntero al nodo anterior.

El uso de punteros permite que la pila crezca dinámicamente según sea necesario, eliminando la restricción de tamaño fijo que tienen los arreglos estáticos. Se implementaron las mismas operaciones básicas: `push`, `pop`, `peek` y funciones de verificación.

Se gestionó correctamente la liberación de memoria con `delete` para evitar fugas de memoria. Los resultados fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

20. PILA CON PUNTEROS (NUEVO TIPO DE DATO)

Se combinó la implementación de la Pila con punteros y un tipo de dato personalizado. Cada nodo almacena un objeto del tipo definido por el usuario y un puntero al siguiente nodo, conformando una estructura enlazada dinámica.

Esta variante demuestra la versatilidad de las pilas enlazadas para almacenar cualquier tipo de dato, independientemente de su complejidad. Se verificó el correcto funcionamiento de todas las operaciones con el tipo personalizado.

Los resultados fueron exportados en los cuatro formatos de salida requeridos.

21. PILA CON LIBRERÍAS (DATO BASE)

En esta práctica se utilizó la librería estándar de C++ (STL) para implementar la Pila mediante el contenedor `std::stack`. Este enfoque aprovecha las implementaciones ya optimizadas y probadas que ofrece la biblioteca estándar.

Se exploraron los métodos proporcionados por `std::stack`: `push()`, `pop()`, `top()`, `empty()` y `size()`. Se comparó la facilidad de uso de la librería estándar frente a las implementaciones manuales de prácticas anteriores.

Los resultados obtenidos fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

22. PILA CON LIBRERÍAS (NUEVO TIPO DE DATO)

Se utilizó `std::stack` de la STL para almacenar un tipo de dato personalizado. Se comprobó que el contenedor estándar es genérico y puede utilizarse con cualquier tipo de dato, incluyendo estructuras y clases definidas por el usuario.

Se implementaron operaciones de inserción y extracción de objetos complejos y se verificó que la información se conservara íntegra durante el proceso.

Los resultados fueron exportados en los cuatro formatos de salida establecidos.

23. COLA CON ARREGLOS (DATO BASE)

En esta práctica se implementó la estructura de datos Cola (Queue) utilizando un arreglo estático con tipos de dato primitivos. La Cola sigue el principio FIFO (First In, First Out), donde el primer elemento en ingresar es el primero en ser atendido.

Se implementaron las operaciones fundamentales: `enqueue()` para insertar al final, `dequeue()` para extraer del frente, y funciones de verificación de estado. Se manejó el índice de frente y de final de la cola dentro del arreglo.

Se atendieron los casos de desbordamiento y vaciado. Los resultados fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

24. COLA CON ARREGLOS (NUEVO TIPO DE DATO)

Esta práctica extiende la Cola con arreglos para trabajar con un tipo de dato personalizado. La estructura del arreglo almacena objetos complejos con múltiples atributos, y las operaciones enqueue y dequeue se adaptan para manejar correctamente estos objetos.

Se verificó el correcto orden de atención siguiendo el principio FIFO al insertar y extraer elementos del tipo definido. Los resultados fueron exportados en los cuatro formatos de salida.

25. COLA CON PUNTEROS (DATO BASE)

Se implementó la Cola utilizando nodos enlazados mediante punteros y tipos de dato primitivos. Cada nodo contiene el dato almacenado y un puntero al siguiente nodo. Se mantuvieron dos punteros: uno apuntando al frente y otro al final de la cola.

La implementación dinámica elimina la limitación de tamaño fijo. Se gestionó correctamente la memoria dinámica para evitar fugas. Los resultados fueron exportados en los cuatro formatos requeridos.

26. COLA CON PUNTEROS (NUEVO TIPO DE DATO)

Se combinó la Cola enlazada con punteros y un tipo de dato personalizado. Cada nodo almacena un objeto del tipo definido por el usuario. Las operaciones de enqueue y dequeue funcionan correctamente con el tipo complejo.

Se comprobó que el orden de atención FIFO se respetara para todos los elementos insertados. Los resultados fueron exportados en los cuatro formatos de salida establecidos.

27. COLA CON LIBRERÍAS (DATO BASE)

Se utilizó el contenedor `std::queue` de la librería estándar de C++ para implementar la Cola con tipos de dato primitivos. Los métodos de la STL como `push()`, `pop()`, `front()` y `back()` simplificaron la implementación en comparación con las versiones manuales.

Los resultados fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

28. COLA CON LIBRERÍAS (NUEVO TIPO DE DATO)

Se empleó `std::queue` para almacenar objetos de un tipo personalizado. Se verificó la compatibilidad del contenedor genérico con tipos complejos y se comprobó el correcto funcionamiento de todas las operaciones de la Cola.

Los resultados fueron exportados en los cuatro formatos de salida requeridos.

29. FILA CON ARREGLOS (DATO BASE)

En esta práctica se implementó la estructura Fila (deque o cola doble) usando arreglos estáticos y datos primitivos. La Fila permite la inserción y extracción de elementos tanto por el frente como por el final, combinando características de la Pila y la Cola.

Se implementaron operaciones para insertar y extraer por ambos extremos, así como funciones de verificación de estado. Se gestionaron correctamente los índices de ambos extremos dentro del arreglo.

Los resultados fueron exportados en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.

30. FILA CON ARREGLOS (NUEVO TIPO DE DATO)

Esta práctica extiende la Fila con arreglos para manejar un tipo de dato personalizado. Las operaciones de inserción y extracción por ambos extremos se adaptan correctamente para trabajar con objetos complejos.

Los resultados fueron exportados en los cuatro formatos de salida establecidos.

31. FILA CON PUNTEROS (DATO BASE)

Se implementó la Fila de forma dinámica utilizando punteros y datos primitivos. La estructura utiliza nodos enlazados con dos punteros por nodo (anterior y siguiente) para permitir la navegación en ambas direcciones.

Se gestionó la memoria dinámica adecuadamente y se validaron las operaciones en ambos extremos. Los resultados fueron exportados en los cuatro formatos requeridos.

32. FILA CON PUNTEROS (NUEVO TIPO DE DATO)

La Fila doblemente enlazada con punteros se implementó para almacenar un tipo de dato personalizado. Las operaciones en ambos extremos funcionan correctamente con objetos complejos y la memoria se gestiona sin fugas.

Los resultados fueron exportados en los cuatro formatos de salida requeridos.

33. FILA CON LIBRERÍAS (DATO BASE)

Se utilizó el contenedor `std::deque` de la STL para implementar la Fila con datos primitivos. Este contenedor permite acceso, inserción y eliminación eficiente en ambos extremos, con una interfaz más sencilla que las implementaciones manuales.

Los resultados fueron exportados en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.

34. FILA CON LIBRERÍAS (NUEVO TIPO DE DATO)

Se empleó `std::deque` para almacenar objetos de un tipo de dato personalizado. Se verificó que el contenedor genérico manejara correctamente los objetos complejos en ambos extremos de la estructura.

Los resultados fueron exportados en los cuatro formatos de salida establecidos.

35. ORDENAMIENTO BUBBLE SORT

En esta práctica se implementó el algoritmo de ordenamiento Bubble Sort (burbuja). Este algoritmo compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto, repitiendo el proceso hasta que la secuencia queda completamente ordenada.

Se analizó la complejidad temporal del algoritmo ($O(n^2)$ en el peor caso) y se comparó su rendimiento con arreglos de distintos tamaños. A pesar de su simplicidad, es eficiente para arreglos pequeños o casi ordenados.

Los resultados del ordenamiento fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

36. ORDENAMIENTO MERGE SORT

Se implementó el algoritmo de ordenamiento Merge Sort (mezcla), basado en el paradigma divide y vencerás. El arreglo se divide recursivamente en dos mitades, cada mitad se ordena de forma independiente y luego ambas mitades se fusionan en orden correcto.

Merge Sort tiene una complejidad temporal de $O(n \log n)$ en todos los casos, lo que lo hace más eficiente que Bubble Sort para arreglos grandes. Se implementaron las funciones de división y fusión por separado para mantener la claridad del código.

Los resultados fueron exportados en los cuatro formatos de salida requeridos.

37. ORDENAMIENTO QUICK SORT

En esta práctica se implementó el algoritmo Quick Sort, otro algoritmo basado en divide y vencerás. Se selecciona un elemento pivote y se reorganiza el arreglo de modo que los elementos menores al pivote queden a su izquierda y los mayores a su derecha. Luego se aplica el mismo proceso recursivamente a cada partición.

Quick Sort tiene una complejidad promedio de $O(n \log n)$, aunque en el peor caso puede degradarse a $O(n^2)$ si el pivote siempre es el elemento menor o mayor. Se implementó la selección del pivote de forma estratégica para minimizar este riesgo.

Los resultados del ordenamiento fueron exportados en los cuatro formatos: `salida.txt`, `salida.csv`, `salida.json` y `salida.xml`.

TERCER PARCIAL

38. IMPLEMENTACIÓN DE GRAFO (NO DIRIGIDO)

En esta práctica se implementó una estructura de datos tipo Grafo no dirigido, diseñada para representar rutas entre distintos lugares mediante conexiones bidireccionales. Cada nodo representa una ubicación identificada por un ID y una etiqueta descriptiva, mientras que las aristas almacenan información adicional como tiempo de recorrido, costo y tipo de conexión.

La estructura fue desarrollada utilizando vectores dinámicos y tipos de dato personalizados, permitiendo administrar tres componentes principales:

- Colección de nodos.
- Colección de aristas.
- Conjunto de conexiones entre pares de nodos.

Cada conexión almacena:

- Nodo inicial.
- Nodo final.
- Nombre de la conexión.
- Tiempo de traslado.
- Costo asociado.

Al tratarse de un grafo no dirigido, las conexiones permiten tránsito en ambos sentidos, por lo que durante el procesamiento las relaciones se consideran equivalentes tanto de ida como de regreso.

Se implementó una función para visualizar completamente la estructura del grafo mostrando:

- Total de nodos registrados.
- Lista de aristas disponibles.
- Conexiones existentes entre nodos.
- Tiempo y costo de cada recorrido.

Como parte del análisis del grafo se incorporaron algoritmos de búsqueda y optimización:

Algoritmo de Dijkstra

Se desarrolló una implementación del algoritmo de Dijkstra para calcular el camino mínimo desde un nodo origen hacia todos los demás nodos del sistema.

El algoritmo realiza:

1. Inicialización de distancias.
2. Selección del nodo no visitado con menor costo.
3. Relajación de conexiones.
4. Reconstrucción del camino óptimo.

El criterio utilizado para optimizar rutas fue el tiempo de recorrido.

Ruta más larga mediante DFS + Backtracking

Además del camino mínimo, se implementó un algoritmo basado en Depth First Search (DFS) con Backtracking para determinar la ruta más larga entre dos nodos.

Este procedimiento:

- Explora todas las rutas posibles.
- Evita ciclos mediante control de nodos visitados.
- Calcula el tiempo acumulado.
- Conserva el recorrido con mayor duración.

Para facilitar el manejo de información externa se desarrolló lectura automática desde archivos:

- XML
- JSON

Las funciones de carga reconstruyen completamente el grafo importando:

- nodos;
- conexiones;
- tiempos;
- costos;
- nombres de aristas.

Antes de cada carga se realiza limpieza de estructuras para evitar duplicación de datos y mantener consistencia.

Adicionalmente se implementó un sistema de generación automática de resultados en múltiples formatos:

- salida.txt
- salida.csv
- salida.json
- salida.xml

Durante el desarrollo surgieron diversos errores técnicos.

Inicialmente aparecieron problemas relacionados con la duplicación de nodos y conexiones después de múltiples cargas desde archivos externos. También se detectaron errores al reconstruir rutas dentro del algoritmo de Dijkstra debido a diferencias entre el identificador lógico del nodo y su posición dentro del vector.

Se identificaron además problemas como:

- referencias inválidas durante recorridos del grafo;
- conexiones inexistentes entre nodos;
- errores al interpretar etiquetas en XML y JSON;
- rutas inaccesibles durante búsqueda;

- acceso fuera de rango en estructuras dinámicas;
- caminos incompletos durante reconstrucción del recorrido.

Para solucionarlos se aplicaron las siguientes medidas:

- limpieza completa del grafo antes de cada importación;
- validación de existencia de nodos;
- control de nodos visitados;
- verificación de índices antes del acceso;
- depuración del algoritmo DFS;
- pruebas con distintos conjuntos de datos;
- validación del proceso de exportación.

Finalmente se verificó el funcionamiento mediante pruebas de carga de archivos, cálculo de rutas mínimas y máximas, recorrido completo del grafo y generación correcta de todos los archivos de salida.

39. IMPLEMENTACIÓN DE DÍGRAFO (DIRIGIDO)

En esta práctica se desarrolló una estructura de datos tipo Dígrafo (Grafo Dirigido) para representar conexiones entre diferentes nodos donde cada recorrido posee una dirección específica de origen hacia destino.

La estructura fue implementada utilizando vectores dinámicos y estructuras personalizadas, organizando la información mediante:

- colección de nodos;
- colección de aristas;
- conjunto de conexiones dirigidas;
- almacenamiento de tiempo y costo de recorrido.

Cada conexión almacena:

- nodo origen;
- nodo destino;
- nombre de la conexión;
- tiempo de recorrido;
- costo asociado.

Se implementó una función para registrar nuevos nodos dentro del sistema mediante un identificador único y una etiqueta descriptiva.

Inserción de conexiones dirigidas

Se desarrolló un procedimiento para agregar conexiones respetando estrictamente la dirección de la ruta.

El sistema verifica además que las aristas no se dupliquen dentro del conjunto almacenado.

Visualización del dígrafo

Se implementó una función de recorrido que permite mostrar:

- nodos existentes;
- aristas registradas;
- conexiones dirigidas;
- tiempo de recorrido;
- costo asociado.

Para el análisis del comportamiento del dígrafo se integraron algoritmos de búsqueda y optimización.

Algoritmo de Dijkstra (dirigido)

Se desarrolló una implementación del algoritmo de Dijkstra adaptada específicamente para dígrafos.

El algoritmo realiza:

1. inicialización de distancias;
2. selección del nodo con menor costo acumulado;
3. relajación únicamente sobre conexiones salientes;
4. reconstrucción del camino óptimo.

También se incorporó soporte para lectura automática de archivos externos mediante:

- XML
- JSON

Durante la carga se reconstruyen automáticamente:

- nodos;
- conexiones;
- tiempos;
- costos;
- nombres de aristas.

Como salida del procesamiento se desarrolló exportación automática hacia:

- salida.txt
- salida.csv
- salida.json
- salida.xml

Durante el desarrollo surgieron diversos problemas técnicos.

Inicialmente aparecieron errores relacionados con el manejo de direcciones dentro del dígrafo, ya que algunas conexiones se estaban interpretando como bidireccionales provocando resultados incorrectos en Dijkstra y DFS.

También se detectaron errores como:

- rutas inaccesibles por orientación incorrecta;
- reconstrucción incompleta del camino óptimo;
- nodos inexistentes durante recorridos;
- errores de lectura de XML y JSON;
- accesos inválidos al vector de nodos;
- ciclos no controlados durante búsqueda profunda;
- duplicación de conexiones después de múltiples importaciones.

Para solucionarlos se implementaron las siguientes medidas:

- validación estricta de dirección origen → destino;
- limpieza previa de estructuras antes de cargar archivos;
- control de nodos visitados;
- validación de índices;
- corrección del algoritmo de relajación en Dijkstra;
- pruebas de recorridos dirigidos;
- verificación de exportación de resultados.

Finalmente se comprobó el funcionamiento mediante pruebas de carga, cálculo de caminos mínimos, búsqueda de rutas largas y generación correcta de salidas en todos los formatos requeridos.

40. IMPLEMENTACIÓN DE ÁRBOL BST

En esta práctica se implementó una estructura de datos Árbol Binario de Búsqueda (BST – Binary Search Tree) utilizando Programación Orientada a Objetos y almacenamiento dinámico en memoria. Cada nodo del árbol almacena información correspondiente a un lugar representado mediante un identificador numérico y una etiqueta descriptiva.

La construcción del árbol se realiza mediante una función de inserción recursiva que organiza automáticamente los elementos siguiendo la propiedad fundamental del BST:

- Los valores menores se almacenan en el subárbol izquierdo.
- Los valores mayores o iguales se almacenan en el subárbol derecho.

Además del árbol principal, el sistema integra estructuras complementarias para almacenar:

- Lista de nodos
- Lista de aristas
- Conjunto de conexiones nodo a nodo
- Información de tiempo y costo de recorrido

Como extensión del proyecto se incorporaron algoritmos de análisis sobre las conexiones almacenadas:

- Algoritmo de Dijkstra, utilizado para calcular el camino mínimo desde un nodo origen hacia el resto del árbol utilizando el tiempo como criterio de optimización.
- Búsqueda DFS (Depth First Search) con Backtracking, utilizada para encontrar la ruta más larga entre dos nodos considerando el costo acumulado del recorrido.

El sistema incorpora lectura de datos externos mediante archivos XML y JSON, permitiendo reconstruir automáticamente toda la estructura del árbol y sus conexiones. Para ello se desarrollaron funciones específicas que interpretan manualmente el contenido de los archivos y convierten la información en objetos internos.

También se implementó un sistema de exportación automática de resultados en múltiples formatos:

- salida.txt
- salida.csv
- salida.json
- salida.xml

Durante el desarrollo surgieron distintos errores técnicos. Inicialmente se detectaron problemas al reconstruir el árbol después de cargar archivos externos, provocando que algunos recorridos no mostraran todos los nodos. También aparecieron errores al asociar correctamente los identificadores con sus posiciones dentro de las estructuras auxiliares utilizadas por Dijkstra.

Adicionalmente se encontraron errores relacionados con:

- inserciones repetidas dentro del BST;
- referencias inválidas durante recorridos recursivos;
- accesos fuera del rango del vector;

- rutas inexistentes durante el cálculo del camino más largo;
- lectura incorrecta de etiquetas y conexiones desde XML y JSON.

Estos problemas se solucionaron mediante:

- reinicialización completa de estructuras antes de cada carga;
- validación de existencia del nodo origen y destino;
- control de nodos visitados para evitar ciclos;
- verificación de índices antes del acceso;
- depuración de funciones recursivas;
- pruebas con diferentes conjuntos de datos.

Finalmente se verificó el funcionamiento mediante pruebas de inserción, recorridos del árbol, cálculo de rutas mínimas y máximas, además de la correcta generación de archivos de salida.

El programa lee los datos de entrada desde datos.xml o datos.json con la misma información de nodos y conexiones. Al ejecutarse, genera automáticamente los resultados de los tres recorridos y los exporta en los cuatro formatos: salida.txt, salida.csv, salida.json y salida.xml.